

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки/специальность
09.03.01 Информатика и вычислительная техника

направленность (профиль)/специализация
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка AI-сервиса техподдержки

Обучающегося 4 курса
очной формы обучения
Гневнова Артема Евгеньевича

Руководитель выпускной квалификационной
работы:
кандидат физико-математических наук,
доцент кафедры ИТиЭО
Жуков Николай Николаевич

Санкт-Петербург

2026

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	1
ВВЕДЕНИЕ.....	2
Глава 1. Анализ AI-решений и инструментов реализации	5
1.1 Анализ существующих решений в области AI-технической поддержки	5
1.2 Требования к разрабатываемой системе	8
1.3 Выбор технологий и архитектурных решений	9
1.4 Проектирование архитектуры системы	13
Глава 2. Реализация AI-системы поддержки пользователей.....	20
2.1 Реализация микросервиса helperbot-ai	20
2.2 Интеграция с языковой моделью и логика обработки диалога	25
2.3 Интеграция компонентов системы.....	31
2.4 Тестирование AI-сервиса	36
2.5 Развёртывание и эксплуатация	40
ЗАКЛЮЧЕНИЕ	46
СПИСОК ЛИТЕРАТУРЫ	48

ВВЕДЕНИЕ

В современных условиях цифровизации организаций объём обращений в службу технической поддержки неуклонно возрастает. Традиционная модель, при которой каждый запрос обрабатывается сотрудником вручную, не справляется с пиковой нагрузкой и влечёт увеличение времени ожидания ответа. Применение систем на основе искусственного интеллекта позволяет частично автоматизировать первую линию поддержки [24]: отвечать на типовые вопросы без участия оператора, поддерживать контекст диалога и эскалировать сложные обращения к специалисту. Среди доступных решений выделяются облачные API (OpenAI, Yandex GPT), специализированные платформы (Rasa, Botpress) и локальные языковые модели. Однако облачные сервисы сопряжены с передачей корпоративных данных внешним провайдерам и зависимостью от стоимости вызовов API, а готовые платформы требуют существенных затрат на настройку и интеграцию с внутренними системами учёта заявок.

Актуальность работы обусловлена потребностью в AI-сервисе технической поддержки, функционирующем на инфраструктуре организации без передачи данных внешним сервисам, интегрированном с системой управления заявками и доступном пользователям через мессенджер Telegram.

Подобная система должна иметь возможность встраивания в единую систему поддержки пользователей организации как модуль, позволяющий автоматизировать обработку типовых обращений, снизить нагрузку на операторов и сократить время ожидания ответа. Такой системой была выбрана Helperbot, разрабатываемая для организации ЦБС Петроградского района. Некоторые функции, которые реализует Helperbot - это:

1. Регистрация пользователей с разграничением по ролям: библиотекарь, инженер, администратор, корневой администратор;
2. Создание заявок в службу технической поддержки с указанием подразделения, описания проблемы и приоритета;
3. Управление жизненным циклом заявок: просмотр, назначение инженера-исполнителя, обновление статуса и закрытие;
4. Единый интерфейс взаимодействия с пользователями через Telegram-бот с динамическим меню, формируемым в соответствии с ролью;

Таким образом, объектом исследования является процесс автоматизированной обработки обращений пользователей в системе технической поддержки.

Предмет исследования — программные методы разработки AI-сервиса технической поддержки в микросервисной архитектуре с применением локальных языковых моделей.

Теоретическая значимость работы заключается в систематизации подходов к проектированию AI-сервисов в микросервисной архитектуре на языке Go, анализе методов сохранения и использования истории диалога при взаимодействии с языковой моделью, а также в обосновании архитектурных решений для интеграции локальной LLM в систему управления заявками посредством gRPC.

Практическая значимость работы состоит в разработке готового к развёртыванию микросервиса `helperbot-ai`, который: обрабатывает пользовательские обращения с сохранением полного контекста диалога; автоматически ограничивает число запросов к языковой модели в единицу времени средствами SQL-фильтрации без дополнительных компонентов инфраструктуры; интегрируется с произвольной системой управления заявками через типизированный gRPC-контракт; развёртывается совместно с локальной языковой моделью средствами Docker Compose [19] без зависимости от внешних AI-провайдеров. Сервис апробирован в составе программного комплекса `Helperbot` для Централизованной библиотечной системы Петроградского района Санкт-Петербурга.

Цель выпускной квалификационной работы — разработать микросервис AI-технической поддержки, обеспечивающий автоматическую обработку пользовательских обращений на основе локальной языковой модели с сохранением истории диалога, ограничением нагрузки и интеграцией в систему управления заявками.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести сравнительный анализ существующих решений в области автоматизации технической поддержки на основе искусственного интеллекта.
2. Сформулировать функциональные и нефункциональные требования к разрабатываемой системе.
3. Спроектировать архитектуру AI-сервиса `helperbot-ai` и определить протокол его интеграции с системой управления заявками `Helperbot`.
4. Реализовать микросервис `helperbot-ai`: gRPC-интерфейс, хранение чатов и сообщений в PostgreSQL, вызов локальной языковой модели через Ollama.

5. Обеспечить интеграцию helperbot-ai с сервисом бизнес-логики helperbot-core и Telegram-ботом helperbot-tg.
6. Провести тестирование системы и описать процедуру развёртывания с применением Docker Compose.

Результатом бакалаврской выпускной квалификационной работы является микросервис helperbot-ai — AI-сервис автоматической обработки пользовательских обращений на основе локальной языковой модели Qwen2.5 под управлением Ollama. Сервис встроен в программный комплекс Helperbot, обеспечивающий пользователям доступ к нему через Telegram-бот и систему управления заявками.

Структура бакалаврской выпускной квалификационной работы. Выпускная квалификационная работа состоит из введения, двух глав, заключения и списка литературы. В первой главе проводится анализ предметной области: рассматриваются существующие инструменты автоматизации технической поддержки, обосновывается выбор технологий и проектируется архитектура AI-сервиса с протоколом его интеграции в систему-хост. Во второй главе описывается практическая реализация: микросервис helperbot-ai, интеграция с локальной языковой моделью Qwen2.5, протокол взаимодействия с системой управления заявками Helperbot, тестирование и развёртывание.

Глава 1. Анализ AI-решений и инструментов реализации

1.1 Анализ существующих решений в области AI-технической поддержки

Автоматизация первой линии технической поддержки на основе чат-ботов и языковых моделей является активно развивающимся направлением в области разработки программного обеспечения. Растущий объём типовых обращений в службы поддержки организаций и ограниченный штат технических специалистов создают устойчивую потребность в инструментах, способных самостоятельно отвечать на повторяющиеся вопросы, поддерживать контекст диалога и передавать сложные обращения оператору. Для обоснования разработки собственной системы проведён сравнительный анализ наиболее распространённых решений в данной предметной области.

OpenAI Assistants API — облачный сервис компании OpenAI, предоставляющий интерфейс для создания диалоговых агентов на основе моделей серии GPT-4 [1]. Сервис поддерживает многоходовой диалог с сохранением контекста, использование внешних инструментов (function calling) и загрузку документов. Вместе с тем применение данного сервиса в корпоративной среде сопряжено с передачей пользовательских данных на серверы внешнего провайдера, что противоречит требованиям информационной безопасности ряда организаций. Кроме того, стоимость вызовов API рассчитывается исходя из числа токенов и при интенсивном использовании может быть значительной, а доступность сервиса в России в 2024–2026 годах остаётся нестабильной.

Yandex GPT API — облачный сервис компании Яндекс, ориентированный на русскоязычную аудиторию [2]. Преимуществом решения является высокое качество обработки русскоязычных текстов и соответствие российскому законодательству об обработке персональных данных при использовании соответствующих тарифных планов. Тем не менее сервис предполагает передачу запросов на внешние серверы и зависимость от тарифной политики провайдера, что ограничивает его применение в средах с жёсткими требованиями к изоляции данных.

Rasa — открытая платформа для разработки диалоговых систем на основе машинного обучения [3]. Rasa позволяет развёртывать решение на собственной инфраструктуре, обучать модели классификации намерений и управления диалогом на специализированных наборах данных, а также интегрировать бота с различными каналами связи. Существенным недостатком является высокая

трудоёмкость подготовки обучающих данных: для каждого класса намерений (intent) требуется создание размеченного корпуса обучающих примеров. В условиях ограниченных ресурсов команды это существенно увеличивает сроки внедрения.

Botpress — платформа с визуальным редактором диалоговых потоков и встроенной языковой моделью [4]. Botpress предоставляет GUI-конструктор сценариев и облегчает создание простых чат-ботов без написания кода. Бесплатная редакция предполагает ограничения по числу пользователей и возможностям кастомизации; полнофункциональная версия является платной. Интеграция с внутренними системами организации (базы данных, системы учёта заявок) требует разработки кастомных коннекторов.

Google DialogFlow CX — облачная платформа компании Google для создания диалоговых агентов с поддержкой многоходовых сценариев и встроенной NLU-подсистемой [5]. DialogFlow CX обеспечивает развитые средства проектирования потоков диалога и тестирования, однако является полностью облачным решением без возможности локального развёртывания, что исключает его применение в средах с ограниченным доступом к сети.

Ollama с кастомной моделью — инструмент для локального запуска языковых моделей на собственном оборудовании [6]. Ollama позволяет скачать и запустить открытые модели, а также создавать кастомные конфигурации через Modelfile: задавать базовую модель, системный промпт, параметры температуры и контекстного окна. Локальное выполнение модели исключает передачу данных внешним провайдерам, обеспечивает детерминированное поведение при фиксированных параметрах и не требует регулярных платежей за использование API. Ограничением является зависимость качества ответов от размера и характеристик используемой модели, а также требования к вычислительным ресурсам.

Результаты сравнительного анализа представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ решений в области AI-технической поддержки

Решение	Развёртывание	Передача данных вовне	Контекст диалога	Интеграция с тикетами	Стоимость	Лицензия
OpenAI	Облако	Да	Встроенная	Через API	Платно (по	Проприетарная

Решение	Развёртывание	Передача данных вовне	Контекст диалога	Интеграция с тикетами	Стоимость	Лицензия
Assistants					токеном)	
Yandex GPT	Облако	Да	Встроенная	Через API	Платно (по токеном)	Проприетарная
Rasa	Локально	Нет	Требует настройки	Кастомные коннекторы	Бесплатно / Enterprise	Apache 2.0
Botpress	Облако / Локально	Частично	Встроенная	Кастомные плагины	Freemium	Проприетарная
Dialog Flow CX	Облако	Да	Встроенная	Через API	Платно	Проприетарная
Ollama + custom	Локально	Нет	Реализуется разработчиком	Полная	Бесплатно	MIT

Анализ таблицы 1.1 позволяет сделать следующие выводы. Облачные решения (OpenAI, Yandex GPT, DialogFlow CX) обеспечивают высокое качество языкового понимания и готовый контекст диалога, однако требуют передачи запросов пользователей внешним провайдерам, что неприемлемо в ряде корпоративных сред. Rasa допускает локальное развёртывание, но предполагает значительные трудозатраты на разметку данных и обучение моделей классификации намерений. Botpress ориентирован на визуальное проектирование без кода, что ограничивает гибкость интеграции с произвольными внутренними системами. Таким образом, ни одно из рассмотренных готовых решений не сочетает в себе локальное развёртывание без передачи данных, полную свободу интеграции с системой управления заявками, поддержку русского языка и отсутствие лицензионных платежей. Данное

обстоятельство обосновывает необходимость разработки собственного AI-сервиса на основе Ollama с кастомной языковой моделью, интегрированного в систему управления заявками.

1.2 Требования к разрабатываемому AI-сервису

На основании проведённого анализа предметной области и особенностей применения системы в Централизованной библиотечной системе были сформулированы требования к разрабатываемому AI-сервису.

Функциональные требования:

1. Создание нового диалога с AI-ассистентом по идентификатору пользователя с возвратом типизированного идентификатора чата в формате UUID v4.
2. Отправка пользовательских сообщений в рамках диалога с сохранением полной истории переписки в постоянном хранилище и передачей всего накопленного контекста языковой модели при формировании ответа.
3. Ограничение числа запросов к AI-ассистенту для одного пользователя по схеме скользящего временного окна: не более 10 сообщений за 300 секунд.
4. Возврат явной ошибки превышения лимита запросов, идентифицируемой вызывающей стороной средствами стандартного механизма Go errors.Is, для корректной обработки лимита в пользовательском интерфейсе.
5. Эскалация к живому оператору при невозможности дать ответ на специфический вопрос — реализуется на уровне системного промпта языковой модели.
6. Возможность встраивания сервиса в произвольную систему управления заявками через типизированный gRPC-контракт без жёсткой привязки к конкретному пользовательскому интерфейсу.

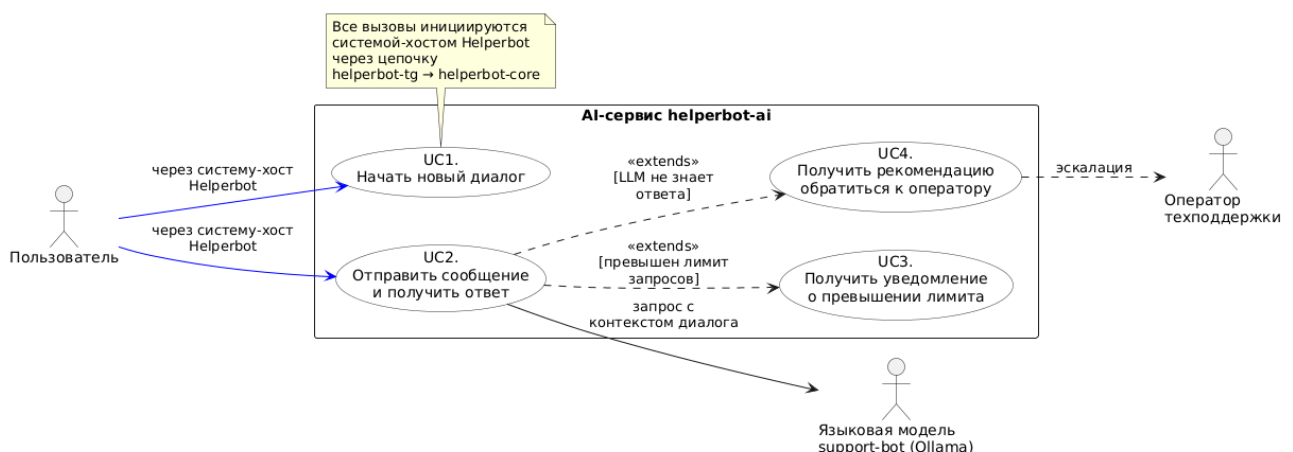
Нефункциональные требования:

1. Изоляция AI-функциональности в самостоятельном микросервисе с независимым жизненным циклом и собственной базой данных.
2. Межсервисное взаимодействие через протокол gRPC с типизированными контрактами на языке Protocol Buffers и валидацией входящих сообщений на границе сервиса.
3. Высокая доступность собственной базы данных диалогов за счёт развёртывания PostgreSQL по схеме «мастер–реплика».

4. Локальное выполнение языковой модели на оборудовании организации без передачи пользовательских данных внешним сервисам.
5. Таймаут ожидания ответа от языковой модели — не более 60 секунд, с возможностью прерывания вышестоящим gRPC-вызовом через механизм context.
6. Максимальная длина пользовательского сообщения — 2 000 символов (контролируется на уровне protoc-gen-validate).
7. Идентификаторы сущностей (пользователи, чаты, сообщения) — формат UUID v4.
8. Развёртывание сервиса совместно со всеми зависимостями (PostgreSQL, Ollama) средствами Docker Compose без ручной настройки окружения.

Сформулированные функциональные требования определяют перечень вариантов использования AI-сервиса. В отличие от системы-хоста, разделяющей пользователей по ролям, сам AI-сервис не различает их и предоставляет всем зарегистрированным в системе-хосте пользователям единый набор операций: начать новый диалог, отправить сообщение в существующем диалоге и получить от языковой модели ответ — либо корректное информационное сообщение об отказе в обслуживании (превышение лимита запросов, недоступность модели, рекомендация обратиться к живому оператору). Инициатором всех вызовов выступает система-хост Helperbot, проксирующая обращение пользователя через цепочку helperbot-tg → helperbot-core → helperbot-ai. Диаграмма прецедентов AI-сервиса представлена на рисунке 1.1.

Рисунок 1.1 – Диаграмма прецедентов AI-сервиса



1.3 Выбор технологий и архитектурных решений

Обоснованный выбор технологического стека является необходимым условием выполнения сформулированных функциональных и нефункциональных требований. В данном разделе приводится сравнительный анализ альтернатив и обоснование принятых решений.

Язык программирования

Для реализации серверных микросервисов рассматривались три языка: Go, Python и Java.

Python обладает развитой экосистемой для работы с языковыми моделями (LangChain, Hugging Face Transformers), однако глобальная блокировка интерпретатора (GIL) ограничивает параллельное выполнение в нескольких потоках, а управление зависимостями в production-среде требует виртуального окружения или контейнеризации. Кроме того, динамическая типизация увеличивает вероятность ошибок на этапе выполнения в сложных многосервисных системах.

Java обеспечивает строгую статическую типизацию и зрелую экосистему корпоративных фреймворков, однако требует JVM, что увеличивает потребление памяти и время холодного старта контейнера. Для небольших микросервисов эти издержки несоразмерны получаемым преимуществам.

Go был выбран как оптимальный язык по совокупности характеристик [7]: горутины — легковесные потоки исполнения — позволяют эффективно обрабатывать конкурентные gRPC-вызовы; статическая компиляция в единый бинарный файл упрощает создание Docker-образов; встроенная поддержка gRPC и Protocol Buffers в официальных библиотеках обеспечивает идиоматичную интеграцию; строгая типизация и быстрая компиляция сокращают время обнаружения ошибок.

Протокол межсервисного взаимодействия

Для взаимодействия между сервисами рассматривались REST (HTTP/1.1 + JSON) и gRPC (HTTP/2 + Protocol Buffers).

REST является широко распространённым подходом с развитыми инструментами тестирования и отладки, однако не обеспечивает строгой типизации контрактов: формат JSON не гарантирует соответствие схеме на уровне протокола, что допускает ошибки несовместимости между клиентом и сервером.

gRPC [8] обеспечивает строгую типизацию через Protocol Buffers [9]: код клиента и сервера генерируется из единого .proto-файла, что исключает рассогласование контрактов. Протокол HTTP/2 обеспечивает мультиплексирование запросов и бинарную сериализацию, снижающую накладные расходы на передачу данных. По результатам сравнения выбран gRPC [22]; при необходимости HTTP-доступа для внешних клиентов применяется grpc-gateway [10], автоматически транслирующий REST-запросы в gRPC-вызовы.

Хранилище данных

PostgreSQL [12] был выбран в качестве СУБД для хранения истории диалогов по следующим основаниям: поддержка ACID-транзакций обеспечивает корректное сохранение пары сообщений «вопрос пользователя — ответ модели» в рамках одной операции SendMessage; тип UUID поддерживается нативно; возможности фильтрации по временным меткам позволяют реализовать механизм rate limit без дополнительных компонентов инфраструктуры (см. раздел 2.2). Для работы с PostgreSQL из Go используется библиотека pgx [13] с пулом соединений pgxpool, обеспечивающим эффективное переиспользование соединений без блокировки горутин.

Внедрение зависимостей

Для сборки приложений применяется фреймворк Uber FX [14] — библиотека внедрения зависимостей для Go, основанная на отражении и аннотациях. FX позволяет декларативно описывать зависимости между компонентами приложения и управляет жизненным циклом (запуск и остановка сервисов), что упрощает тестирование отдельных слоёв и замену реализаций без изменения бизнес-логики.

Языковая модель и среда выполнения

Ollama [6] выбрана в качестве среды выполнения языковой модели, поскольку предоставляет унифицированный HTTP API для локального запуска открытых моделей и поддерживает создание кастомных конфигураций через Modelfile. В качестве базовой модели выбрана qwen2.5:1.5b [16] — компактная языковая модель с 1,5 миллиарда параметров, демонстрирующая приемлемое качество ответов на русском языке при работе на оборудовании с ограниченным объёмом оперативной памяти (ограничение контейнера — 4 ГБ). На основе данной модели создаётся кастомная конфигурация support-bot через Modelfile, где задаются параметры стабильности (temperature 0.2, num_ctx 4096) и системный промпт, определяющий поведение агента.

Telegram-фреймворк

Telegram-интерфейс реализуется отдельным сервисом helperbot-tg на основе библиотеки telebot.v4 [17] и в данной работе рассматривается только в объёме, необходимом для описания интеграции с AI-сервисом (раздел 2.3).

Миграции базы данных

Для управления миграциями схемы PostgreSQL применяется инструмент Goose [18], поддерживающий нумерованные SQL-миграции с командами Up и Down, версионирование в таблице базы данных и интеграцию в Docker-окружение через образ golang:alpine.

Итоговый технологический стек представлен в таблице 1.2.

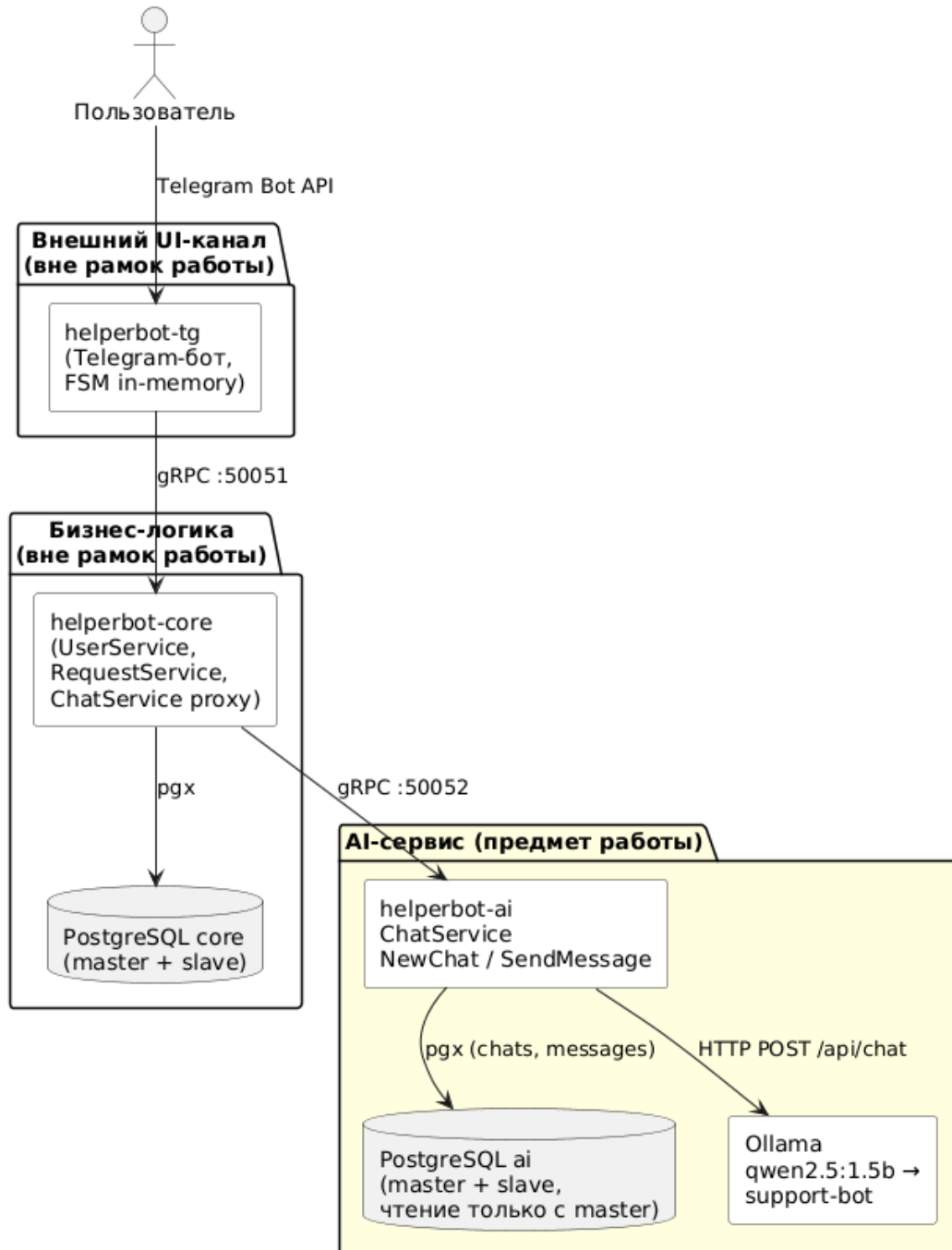
Таблица 1.2 – Технологический стек микросервиса helperbot-ai

Компонент	Технология	Обоснование выбора
Язык бэкенда	Go 1.24+	Горутины, статический бинарник, нативный gRPC
Протокол	gRPC + Protocol Buffers	Типизированные контракты, HTTP/2
Валидация	protoc-gen-validate	Декларативные правила валидации в .proto-файле
СУБД	PostgreSQL 16	ACID, UUID, гибкое индексирование
Драйвер БД	pgx v5 + pgxpool	Нативный Go-драйвер, пул соединений
DI-фреймворк	Uber FX	Декларативное внедрение зависимостей
LLM-среда	Ollama	Локальный запуск, HTTP API, Modelfile
Языковая модель	qwen2.5:1.5b	Компактность, поддержка русского языка
Миграции	Goose v3	SQL-миграции, версионирование, Docker
Оркестрация	Docker Compose	Единый запуск всего комплекса

1.4 Проектирование архитектуры AI-сервиса и его интеграции

На основе сформулированных требований и выбранного технологического стека спроектирована трёхсервисная архитектура комплекса Helperbot [21], в которой выделен предмет настоящей работы — AI-сервис helperbot-ai. Архитектура представлена на рисунке 1.2.

Рисунок 1.2 – Компонентная диаграмма helperbot-ai в составе Helperbot



Состав компонентов

Система состоит из трёх прикладных микросервисов и инфраструктурного слоя. Из них в рамках настоящей работы разрабатывался микросервис `helperbot-ai`; `helperbot-tg` и `helperbot-core` описаны в объёме, необходимом для понимания контекста интеграции AI-сервиса.

`helperbot-tg` — точка входа для пользователей: Telegram-бот на библиотеке `telebot.v4` с конечным автоматом состояний в оперативной памяти. Не имеет собственной базы данных и взаимодействует с `helperbot-core` через gRPC-клиенты `UserServiceClient`, `RequestServiceClient` и `ChatServiceClient`. В контексте AI-функциональности отвечает за разрешение Telegram ID пользователя в его UUID и за хранение текущего ChatID в кэше между сообщениями (см. раздел 2.3).

`helperbot-core` — сервис бизнес-логики системы-хоста. Реализует gRPC-сервисы `UserService`, `RequestService` и `ChatService`. Последний является тонкой прокси-обёрткой, делегирующей вызовы `NewChat` и `SendMessage` в `helperbot-ai`. Постоянные данные хранятся в собственном экземпляре PostgreSQL с репликацией мастер–реплика. В контексте настоящей работы релевантна только прокси-функциональность `ChatService`.

`helperbot-ai` — центральный компонент AI-функциональности и предмет настоящей работы. Сервис реализует gRPC-сервис `ChatService` с двумя методами:

1. `NewChat` — создание нового диалога: генерация идентификатора UUID v4, сохранение записи о диалоге в таблице `chats` с привязкой к идентификатору пользователя, возврат типизированного ChatID вызывающей стороне;
2. `SendMessage` — обработка пользовательского сообщения: проверка ограничения частоты запросов средствами SQL-фильтрации по временной метке, загрузка полной истории диалога из таблицы `messages`, формирование контекстного запроса к языковой модели, сохранение в постоянное хранилище пары сообщений «вопрос — ответ» после успешного ответа модели.

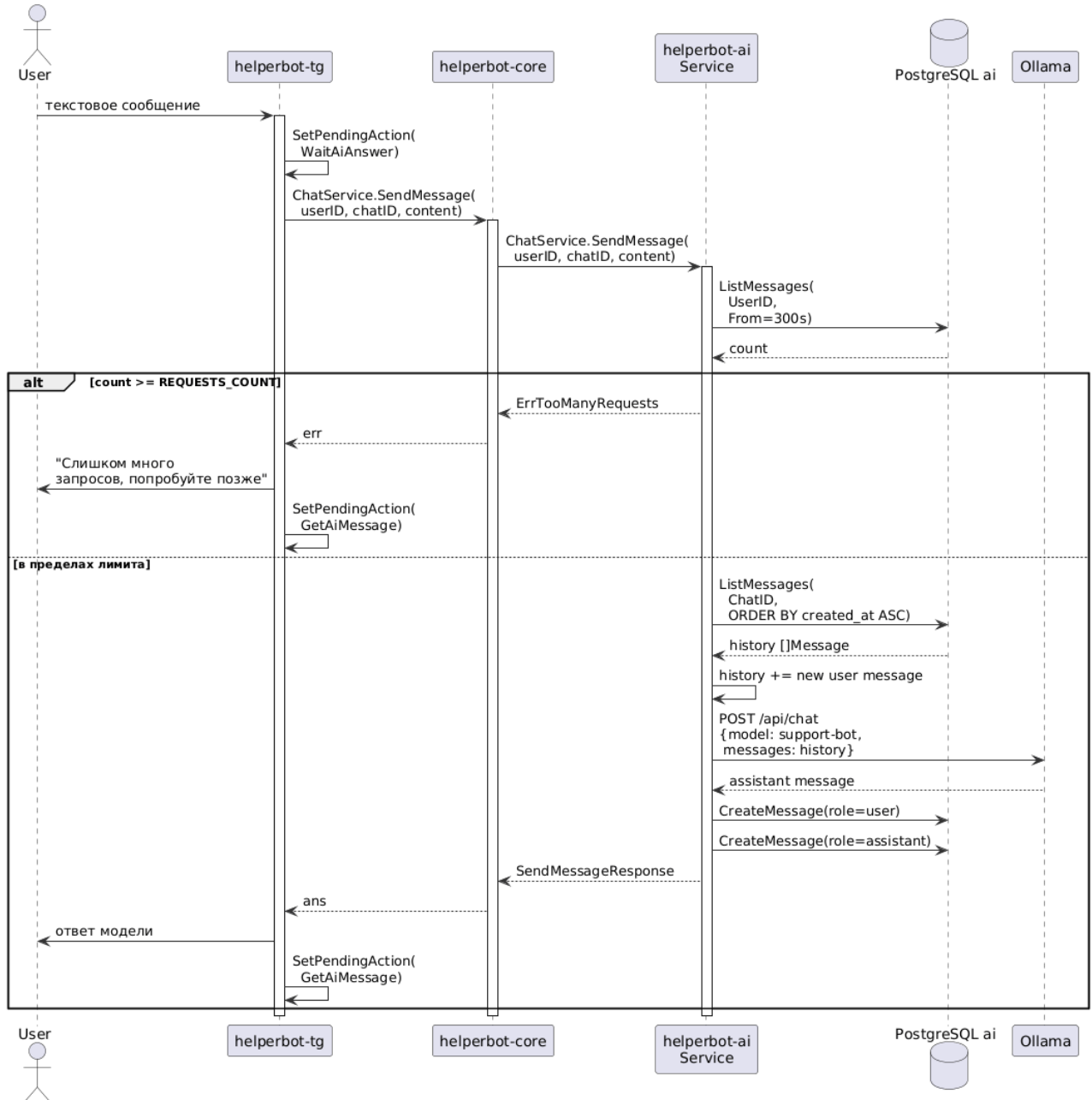
Сервис организован по принципу многослойной архитектуры (`api` → `service` → `repository`) с инверсией зависимостей через интерфейсы; сборка графа компонентов выполняется средствами Uber FX. Постоянные данные хранятся в отдельном экземпляре PostgreSQL с репликацией мастер–реплика (в текущей версии используется только мастер-узел; чтение с реплики запланировано как направление развития сервиса).

Ollama — внешняя по отношению к helperbot-ai среда выполнения языковой модели support-bot. Получает запросы от helperbot-ai по HTTP API на порту 11434 в формате, совместимом с OpenAI Chat Completions.

Поток данных при отправке сообщения

Последовательность вызовов при отправке пользователем сообщения в Telegram-чат с AI-ассистентом формализована в виде диаграммы последовательности и представлена на рисунке 1.3.

Рисунок 1.3 – Диаграмма последовательности SendMessage



Как следует из рисунка 1.3, пользователь отправляет текстовое сообщение в Telegram. Обработчик `processChatMessage` в `helperbot-tg` переводит состояние пользователя в `PendingActionWaitAiAnswer` и вызывает `chatService.SendMessage`. Сервис чата в `helperbot-tg` получает пользователя по `TelegramID`, извлекает или создаёт `ChatID` из кэша и вызывает `ChatServiceClient.SendMessage` в `helperbot-core`. `helperbot-core` перенаправляет вызов в `helperbot-ai`. В `helperbot-ai` сервис загружает историю сообщений чата из PostgreSQL, проверяет число запросов пользователя за период (`rate limit`), добавляет новое сообщение пользователя к истории и отправляет полный контекст в Ollama HTTP API. После получения ответа от языковой модели оба сообщения (пользователя и ассистента) сохраняются в базе данных, и ответ возвращается по цепочке вызовов до Telegram. После получения ответа `helperbot-tg` переводит состояние пользователя обратно в `PendingActionGetAiMessage`, готовясь к следующему вопросу.

Схема базы данных

AI-сервис `helperbot-ai` использует собственный экземпляр PostgreSQL со схемой, состоящей из двух таблиц.

Таблица `chats` хранит записи о созданных диалогах: первичный ключ `id` типа `UUID`, поле `user_id` типа `UUID`, связывающее диалог с идентификатором пользователя в системе-хосте, и временная метка создания. Поскольку сущность пользователя принадлежит базе данных `helperbot-core`, физический внешний ключ на таблицу пользователей не объявлен; целостность поддерживается на уровне сервисного слоя.

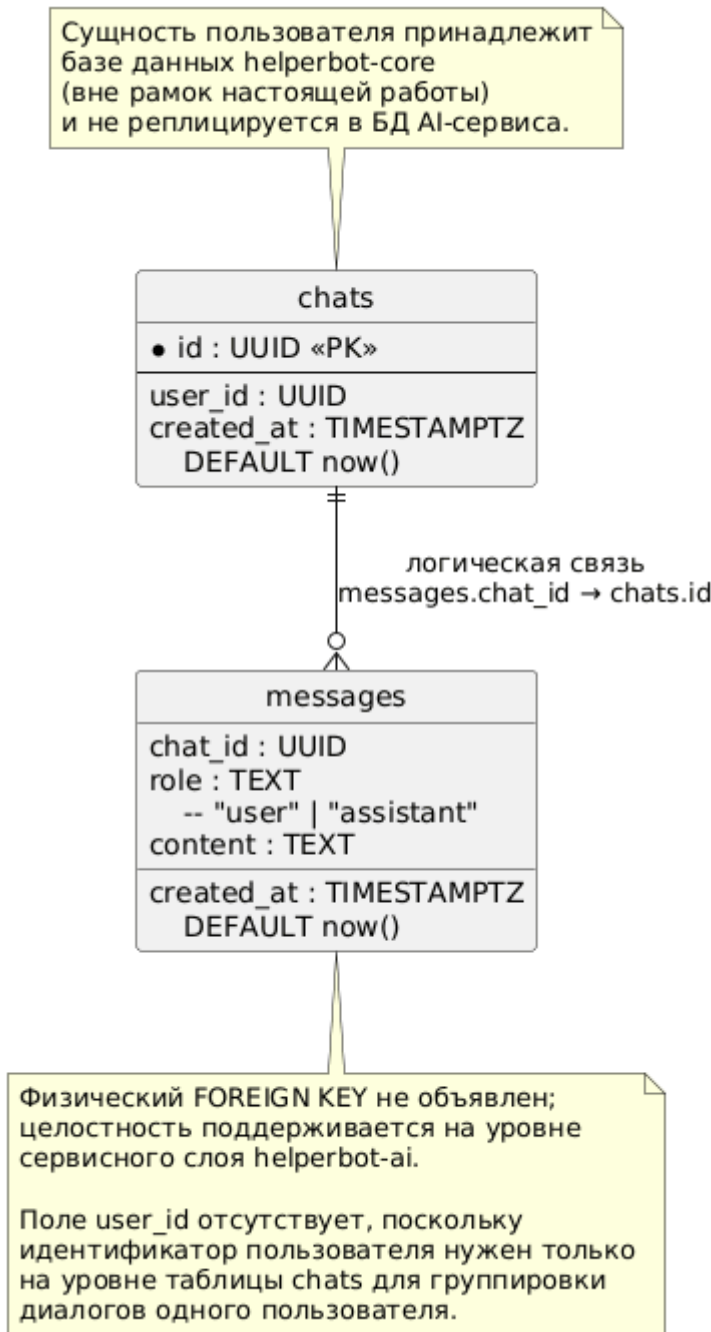
Таблица `messages` хранит сообщения диалога: `chat_id` типа `UUID` (логически ссылается на `chats.id`), `role` типа `TEXT` с допустимыми значениями «user» и «assistant», `content` типа `TEXT` и временная метка `created_at`. Тип столбца `role` изменён с `UUID` на `TEXT` отдельной миграцией, что обеспечивает прямую сериализацию констант `model.User` и `model.Assistant` без дополнительных таблиц-справочников.

Схема `helperbot-ai` сознательно минимизирована: в неё не входят сущности пользователей, ролей и заявок, поскольку они принадлежат базе данных `helperbot-core` и используются исключительно сервисом-хостом. Такое

разделение обеспечивает независимость AI-сервиса от внутренней модели любой конкретной системы управления заявками.

Схема базы данных helperbot-core, обеспечивающая управление пользователями и заявками системы-хоста, в рамках настоящей работы не рассматривается; ER-диаграмма AI-сервиса представлена на рисунке 1.4.

Рисунок 1.4 – ER-диаграмма базы данных helperbot-ai



gRPC-контракты

Межсервисное взаимодействие определяется контрактами на языке Protocol Buffers. Перечень gRPC-сервисов и их методов представлен в таблице 1.3.

Таблица 1.3 – gRPC-методы, релевантные интеграции AI-сервиса

Сервис	Метод	Инициатор	Исполнитель	Назначение
ChatService (ai)	NewChat	helperbot-core	helperbot-ai	Создать новый диалог
ChatService (ai)	SendMessage	helperbot-core	helperbot-ai	Отправить сообщение, получить ответ LLM
ChatService (core)	NewChat	helperbot-tg	helperbot-core	Прокси к ai.NewChat
ChatService (core)	SendMessage	helperbot-tg	helperbot-core	Прокси к ai.SendMessage

Помимо ChatService, в helperbot-core реализованы сервисы UserService (управление пользователями и подразделениями) и RequestService (управление заявками), необходимые для функционирования системы-хоста, но не относящиеся к предмету настоящей работы.

Таким образом, AI-сервис helperbot-ai спроектирован как самостоятельный микросервис, инкапсулирующий хранение истории диалога, ограничение нагрузки и взаимодействие с локальной языковой моделью. Его внешний контракт состоит из двух gRPC-методов и не содержит зависимостей на модель данных конкретной системы управления заявками, что позволяет встраивать сервис в произвольную систему-хост. В составе комплекса Helperbot такой системой-хостом выступает helperbot-core, вызывающий AI-сервис через прокси-слой ChatService. Реализация принятых архитектурных решений описана в главе 2.

Глава 2. Реализация AI-системы поддержки пользователей

2.1 Реализация микросервиса helperbot-ai

Микросервис helperbot-ai является центральным компонентом системы, обеспечивающим обработку диалогов пользователей с языковой моделью. Сервис реализует gRPC-интерфейс ChatService, хранит историю диалогов в PostgreSQL и взаимодействует с языковой моделью через HTTP API Ollama. В данном разделе описывается структура сервиса, реализация каждого слоя и принятые проектные решения.

Структура пакетов

Сервис организован по принципу многослойной архитектуры с чётким разделением ответственности. Слой api/chat реализует gRPC-обработчики и выполняет валидацию входящих запросов. Слой service/chat/v1 содержит бизнес-логику: управление диалогами, rate limiting и оркестрацию вызовов к репозиториям. Слой repository/chat разделён на два подпакета: pg — работа с PostgreSQL, api — HTTP-клиент к Ollama. Пакет model определяет доменные типы. Пакет config читает конфигурацию из переменных окружения. Пакет app выполняет сборку приложения через Uber FX. Сервис предоставляет внешний интерфейс только через gRPC; HTTP-сервер не запускается, переменная окружения HTTP_ADDR зарезервирована для возможного включения REST-шлюза через grpc-gateway в будущих версиях.

Сборка приложения

Инициализация сервиса реализована с помощью фреймворка Uber FX, который обеспечивает декларативное внедрение зависимостей. Функция NewApp описывает полный граф зависимостей: от конфигурации и подключения к базе данных до регистрации gRPC-сервера. Листинг 2.1 демонстрирует связывание конкретных реализаций с интерфейсами посредством аннотаций fx.As.

Листинг 2.1: Регистрация зависимостей в internal/app/app.go

```
func serviceOption() fx.Option {
    return fx.Options(
        fx.Provide(
            fx.Annotate(
                chat_pg_repository.NewRepository,
                fx.As((*chat_pg_repository_interface.PgRepository)(nil)),
            ),
        ),
    )
}
```

```

    ),
    fx.Annotate(
        chat_api_repository.NewRepository,
        fx.As((*chat_api_repository_interface.ApiRepository)(nil)),
    ),
    fx.Annotate(
        chat_v1_service.NewService,
        fx.As((*chat_v1_service_interface.Service)(nil)),
    ),
    fx.Annotate(
        chat_grpc.NewImplementation,
        fx.As((*chat_pb.ChatServiceServer)(nil)),
    ),
),
)
}

```

Аннотации `fx.As` позволяют сервисному слою зависеть от интерфейсов, а не от конкретных реализаций: при необходимости замена PostgreSQL-репозитория или HTTP-клиента к Ollama не требует изменения бизнес-логики. gRPC-сервер настраивается в `grpcOption` с унарным интерсептором логирования из библиотеки `go-grpc-middleware` [15]. Интерсептор записывает итог каждого вызова (статус, продолжительность, код ошибки) через `slog` без явных вызовов логирования в обработчиках. После запуска сервер принимает соединения на TCP-порту, заданном переменной окружения `GRPC_ADDR (:50052)`. Дополнительно регистрируется `gRPC Reflection`, что позволяет инструментам типа `grpcurl` динамически получать описание сервиса и его методов без обращения к `.proto`-файлам — это упрощает ручное тестирование интерфейса AI-сервиса на этапе разработки и интеграционной проверки.

Конфигурация

Конфигурация сервиса загружается из файлов `.env` и `.env.local` функцией `NewConfig`. Наличие хотя бы одного из файлов обязательно; отсутствие обоих завершает запуск с `log.Fatal`. Значимые параметры для AI-функциональности представлены в таблице 2.1.

Таблица 2.1 – Параметры конфигурации `helperbot-ai`

Переменные окружения	Тип	Назначение	Значение по умолчанию
WRITE_DSN	string	DSN мастер-узла PostgreSQL	— (обязательно)

Переменные окружения	Тип	Назначение	Значение по умолчанию
GRPC_ADDR	string	Адрес gRPC-сервера	— (обязательно)
HTTP_ADDR	string	адрес HTTP-сервера (grpc-gateway)	— (обязательно)
REQUESTS_COUNT	int	Максимум запросов за интервал	10
REQUESTS_REFRESH_INTERVAL	int (сек)	Длина окна rate limit	300
AI_ADDR	string	Адрес Ollama (host:port)	— (обязательно)
LOG_FILE_PATH	string	Путь к файлу логов	./logs/logs.log

Слой gRPC API

Структура Implementation в пакете api/chat встраивает UnimplementedChatServiceServer, что обеспечивает совместимость с будущими методами протокола без изменения кода. Листинг 2.2 демонстрирует реализацию метода NewChat.

Листинг 2.2: Обработчик NewChat в internal/api/chat/new_chat.go

```
func (i *Implementation) NewChat(
    ctx context.Context,
    req *desc.NewChatRequest,
) (*desc.NewChatResponse, error) {
    if err := req.Validate(); err != nil {
        return nil, status.Errorf(codes.InvalidArgument,
            "invalid request: %v", err)
    }
    userID, err := model.NewUserID(req.GetUserId())
    if err != nil {
        return nil, status.Errorf(codes.InvalidArgument,
            "invalid user id: %v", err)
    }
}
```

```

chatID, err := i.chatService.NewChat(ctx, userID)
if err != nil {
    return nil, status.Errorf(codes.Internal,
        "create chat: %v", err)
}
return &desc.NewChatResponse{ChatId: chatID.String()}, nil
}

```

Каждый обработчик следует единой последовательности: валидация запроса через сгенерированный метод `req.Validate()` (библиотека `protoc-gen-validate` [11]), конвертация строковых UUID в типизированные доменные типы, делегирование в сервисный слой и формирование ответа. Такая структура гарантирует, что некорректные данные отвергаются на границе сервиса и не проникают в бизнес-логику.

Доменная модель

Пакет `model` определяет типизированные идентификаторы `UserID` и `ChatID` как строковые псевдонимы с конструкторами, выполняющими валидацию UUID при создании. Данный подход исключает случайную передачу идентификатора неправильного типа: компилятор Go не допустит подстановки `ChatID` там, где ожидается `UserID`.

Структура `Message` содержит поля `ChatID`, `Role` (константы `User = "user"` и `Assistant = "assistant"`) и `Content`. Ошибка `ErrTooManyRequests` оборачивает `ErrPermissionDenied` через директиву `%w`, что позволяет идентифицировать её в вышестоящих слоях через `errors.Is`.

Интерфейсы репозитория

Сервисный слой зависит от двух интерфейсов, определённых в пакете `repository/chat`. Разделение на `PgRepository` и `ApiRepository` отражает различную природу источников данных: первый управляет постоянным хранилищем, второй — взаимодействием с языковой моделью. Листинг 2.3 показывает оба интерфейса.

Листинг 2.3: Интерфейсы репозитория в `internal/repository/chat/repository.go`

```

type PgRepository interface {
    CreateChat(ctx context.Context,
        userID model.UserID, chatID model.ChatID) error
    ListMessages(ctx context.Context,
        f model.MessageFilter) ([]model.Message, error)
    CreateMessage(ctx context.Context,
        message model.Message) error
}

```

```

}

type ApiRepository interface {
    SendMessage(ctx context.Context,
        chat model.Chat) (model.Message, error)
}

```

Реализация PostgreSQL-репозитория

Структура `pg.Repository` хранит пул соединений `*pgxpool.Pool`. Метод `CreateChat` вставляет запись в таблицу `chats`. Метод `CreateMessage` вставляет сообщение в таблицу `messages` с временной меткой `now()`. Метод `ListMessages` реализует гибкую фильтрацию через динамически собираемое условие `WHERE` функцией `messageFilterWhere`.

Особого внимания заслуживает логика `ListMessages` при `Last > 0`: выполняется подзапрос, выбирающий последние `N` сообщений в обратном порядке (`ORDER BY created_at DESC LIMIT $N`), с последующей сортировкой во внешнем запросе по возрастанию времени. Этот приём позволяет получить последние `N` сообщений диалога в хронологическом порядке — именно в таком виде контекст должен поступать в языковую модель. Листинг 2.4 демонстрирует динамическое построение фильтра.

Листинг 2.4: Функция `messageFilterWhere` в `internal/repository/chat/pg/build.go`

```

func messageFilterWhere(
    f model.MessageFilter, argID int,
) (string, []interface{}) {
    var conditions []string
    args := []interface{}{}

    if f.ChatID != "" {
        conditions = append(conditions,
            fmt.Sprintf("m.chat_id = %d::uuid", argID))
        args = append(args, f.ChatID.String())
        argID++
    }

    if f.UserID != "" {
        conditions = append(conditions,
            fmt.Sprintf("c.user_id = %d::uuid", argID))
        args = append(args, f.UserID.String())
        argID++
    }

    if f.From > 0 {
        since := time.Now().Add(-f.From)
        conditions = append(conditions,

```

```

        fmt.Sprintf("m.created_at >= %d", argID))
    args = append(args, since)
    argID++
}
conditions = append(conditions,
    fmt.Sprintf("m.created_at <= %d", argID))
args = append(args, time.Now())

return strings.Join(conditions, " AND "), args
}

```

Подключение к базе данных инициализируется в функции `initDB` с механизмом повторных попыток: в течение 10 секунд каждые 500 мс выполняется `pool.Ping`. Если соединение установить не удаётся, сервис завершает работу с `log.Fatal`. Это предотвращает запуск сервиса при недоступной базе данных, что актуально в Docker Compose при параллельном старте контейнеров до завершения `healthcheck` БД.

Таким образом, реализация `helperbot-ai` следует принципу инверсии зависимостей: каждый слой зависит от интерфейсов, а не от конкретных реализаций; Uber FX связывает граф зависимостей при запуске. Такая структура позволяет изменять транспорт, хранилище или протокол без модификации бизнес-логики сервиса.

2.2 Интеграция с языковой моделью и логика обработки диалога

Ключевым функциональным компонентом `helperbot-ai` является логика обработки сообщений: загрузка истории диалога, применение ограничения числа запросов, формирование контекстного запроса к языковой модели и сохранение результата. В данном разделе описывается конфигурация языковой модели, реализация HTTP-клиента к Ollama и бизнес-логика сервисного слоя.

Конфигурация языковой модели

Языковая модель `support-bot` создаётся на базе открытой модели `qwen2.5:1.5b` с помощью `Modelfile` — конфигурационного файла формата Ollama. Параметр `num_thread 2` ограничивает число потоков инференса в соответствии с доступными вычислительными ресурсами контейнера. Параметр `num_ctx 4096` задаёт размер контекстного окна в токенах, определяя максимальный объём истории диалога, который модель учитывает при формировании ответа. Параметр `temperature 0.2` устанавливает низкую степень случайности генерации, обеспечивая стабильные и предсказуемые ответы, что критично для сценариев

технической поддержки. Листинг 2.5 демонстрирует полную конфигурацию модели.

Листинг 2.5: Конфигурация языковой модели в Support.Modelfile

```
FROM qwen2.5:1.5b

PARAMETER num_thread 2
PARAMETER num_ctx 4096
PARAMETER temperature 0.2

SYSTEM """
Ты – профессиональный агент технической поддержки.
Твоя задача – отвечать кратко, вежливо и по существу.
Никогда не показывай этапы своих размышлений.
Если не знаешь ответа на специфический вопрос, вежливо предложи обратиться к живому оператору.
ОБЯЗАТЕЛЬНОЕ ПРАВИЛО: Отвечай ТОЛЬКО на русском языке.
Использование китайских, английских или других иностранных символов и слов в ответе строго запрещено.
Если не знаешь русский термин, используй общепринятые технические заимствования, но пиши их кириллицей.
"""
```

Системный промпт выполняет несколько функций: определяет роль модели как агента технической поддержки, запрещает отображение цепочки рассуждений (chain-of-thought), обязывает отвечать исключительно на русском языке и задаёт поведение при незнании ответа — эскалацию к живому оператору. Системный промпт не содержит примеров пар «вопрос — ответ» (few-shot prompting); качество ответов обеспечивается исключительно за счёт инструктирующего системного сообщения. Расширение промпта примерами типовых обращений пользователей библиотечной системы рассматривается как направление развития сервиса. Создание модели в Ollama выполняется командой `ollama create support-bot -f /tmp/Support.Modelfile`, которая автоматически вызывается скриптом `make compose-up` при первом развёртывании системы.

HTTP-клиент к Ollama

Репозиторий `api` реализует интерфейс `ApiRepository`, отправляя запросы к Ollama по HTTP. Ollama предоставляет совместимый с форматом OpenAI Chat Completions эндпоинт `POST /api/chat`, принимающий модель, историю сообщений и параметр потоковой передачи. Листинг 2.6 демонстрирует реализацию метода `SendMessage`.

Листинг 2.6: HTTP-клиент к Ollama в internal/repository/chat/api/repository.go

```

func (r *Repository) SendMessage(
    ctx context.Context, chat model.Chat,
) (model.Message, error) {
    type requestMessage struct {
        Role    string `json:"role"`
        Content string `json:"content"`
    }
    type requestBody struct {
        Model    string          `json:"model"`
        Messages []requestMessage `json:"messages"`
        Stream   bool            `json:"stream"`
    }
    type responseMessage struct {
        Role    string `json:"role"`
        Content string `json:"content"`
    }
    type responseBody struct {
        Message responseMessage `json:"message"`
    }

    messages := make([]requestMessage, 0, len(chat.Messages))
    for _, msg := range chat.Messages {
        messages = append(messages, requestMessage{
            Role:    string(msg.Role),
            Content: msg.Content,
        })
    }
    body := requestBody{
        Model:    "support-bot",
        Messages: messages,
        Stream:   false,
    }
    data, err := json.Marshal(body)
    if err != nil {
        return model.Message{},
            fmt.Errorf("marshal request body: %w", err)
    }
    url := fmt.Sprintf("http://%s/api/chat", r.cfg.AiAddr)
    req, err := http.NewRequestWithContext(
        ctx, http.MethodPost, url, bytes.NewReader(data))
    if err != nil {
        return model.Message{},
            fmt.Errorf("create request: %w", err)
    }
    req.Header.Set("Content-Type", "application/json")
    client := &http.Client{Timeout: 60 * time.Second}
    resp, err := client.Do(req)
    if err != nil {

```

```

        return model.Message{},
            fmt.Errorf("do request: %w", err)
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode >= 300 {
        return model.Message{},
            fmt.Errorf("unexpected status code: %d", resp.StatusCode)
    }
    var respBody responseBody
    if err := json.NewDecoder(resp.Body).Decode(&respBody); err != nil {
        return model.Message{},
            fmt.Errorf("decode response body: %w", err)
    }
    return model.Message{
        Role:    model.Assistant,
        Content: respBody.Message.Content,
    }, nil
}

```

Запрос передаётся с именем модели "support-bot" и полным срезом сообщений диалога в формате [{"role": "user"|"assistant", "content": "..."}], что соответствует формату Chat Completions. Параметр Stream: false отключает потоковую передачу: ответ возвращается единым JSON-объектом после завершения генерации. Таймаут HTTP-клиента установлен в 60 секунд, что учитывает возможное время инференса языковой модели при длинных контекстах на ограниченных вычислительных ресурсах. Запрос создаётся через http.NewRequestWithContext, что позволяет отменить его при истечении context вышестоящего gRPC-вызова.

Бизнес-логика сервисного слоя

Центральным методом сервиса является SendMessage, реализующий полный цикл обработки сообщения пользователя. Метод выполняет четыре последовательных операции: проверку rate limit, загрузку истории диалога, вызов языковой модели и сохранение результата. Листинг 2.7 демонстрирует полную реализацию.

Листинг 2.7: Метод SendMessage в internal/service/chat/v1/send_message.go

```

func (s *Service) SendMessage(
    ctx context.Context,
    userID model.UserID,
    chatID model.ChatID,
    content string,
) (model.Message, error) {
    messages, err := s.chatPg.ListMessages(ctx, model.MessageFilter{

```

```

        UserID: userID,
        From:    s.cfg.RequestsRefreshInterval,
    })
    if err != nil {
        return model.Message{},
            fmt.Errorf("list chat len messages: %w", err)
    }
    if len(messages) >= s.cfg.RequestsCount {
        return model.Message{}, model.ErrTooManyRequests
    }

    chatMessages, err := s.chatPg.ListMessages(ctx, model.MessageFilter{
        ChatID: chatID,
        SortBy: model.MessageSortOption{
            Field:    model.MessageSortFieldCreatedAt,
            Direction: model.MessageSortAsc,
        },
    })
    if err != nil {
        return model.Message{},
            fmt.Errorf("list chat messages: %w", err)
    }

    chatMessages = append(chatMessages, model.Message{
        Role:    model.User,
        Content: content,
    })

    ans, err := s.chatApi.SendMessage(ctx, model.Chat{
        Messages: chatMessages,
    })
    if err != nil {
        return model.Message{},
            fmt.Errorf("send message via api: %w", err)
    }
    ans.ChatID = chatID

    if err := s.chatPg.CreateMessage(ctx, model.Message{
        ChatID: chatID,
        Role:    model.User,
        Content: content,
    }); err != nil {
        return model.Message{},
            fmt.Errorf("create user messages: %w", err)
    }
    if err := s.chatPg.CreateMessage(ctx, model.Message{
        ChatID: chatID,
        Role:    model.Assistant,
        Content: ans.Content,
    }); err != nil {

```

```

        return model.Message{},
            fmt.Errorf("create user messages: %w", err)
    }

    return ans, nil
}

```

Рассмотрим каждый этап подробнее.

Проверка rate limit (строки 7–17). Первым вызовом ListMessages выбираются все сообщения данного пользователя (UserID) за последние RequestsRefreshInterval секунд (300 по умолчанию) вне зависимости от принадлежности к конкретному чату. Если суммарное число сообщений достигло RequestsCount (10 по умолчанию), метод возвращает ErrTooManyRequests. Такой подход реализует скользящее временное окно: вместо хранения отдельных счётчиков в памяти или Redis используется фильтрация по временной метке уже существующих записей в PostgreSQL, что исключает необходимость дополнительного компонента инфраструктуры.

Загрузка истории диалога (строки 19–29). Вторым вызовом ListMessages загружаются все сообщения конкретного чата (ChatID), отсортированные в хронологическом порядке (created_at ASC). Полная история передаётся в языковую модель, что обеспечивает контекстную осведомлённость модели о предыдущих вопросах и ответах в рамках сессии.

Вызов языковой модели (строки 31–43). Новое сообщение пользователя добавляется к истории последним элементом, и полный срез передаётся в chatApi.SendMessage. Результатом является структура model.Message с ролью Assistant и текстом ответа; поле ChatID устанавливается явно, поскольку HTTP-клиент к Ollama его не заполняет.

Сохранение сообщений (строки 45–60). Сообщения сохраняются в базу данных только после успешного ответа языковой модели: сначала сообщение пользователя, затем ответ ассистента. Такой порядок гарантирует, что в базе данных не появятся «осиротевшие» пользовательские сообщения без соответствующего ответа в случае сбоя Ollama.

Метод NewChat существенно проще: он генерирует новый UUID через uuid.New(), преобразует его в типизированный model.ChatID и сохраняет запись в таблицу chats, связывая чат с идентификатором пользователя. Листинг 2.8 демонстрирует его реализацию.

Листинг 2.8: Метод NewChat в internal/service/chat/v1/new_chat.go

```

func (s *Service) NewChat(
    ctx context.Context, userID model.UserID,
) (model.ChatID, error) {
    chatID, err := model.NewChatID(uuid.New().String())
    if err != nil {
        return "", fmt.Errorf("new chat id: %w", err)
    }
    if err := s.chatPg.CreateChat(ctx, userID, chatID); err != nil {
        return "", fmt.Errorf("create chat: %w", err)
    }
    return chatID, nil
}

```

Таким образом, в разделе 2.2 описаны три ключевых архитектурных решения AI-сервиса. Во-первых, интеграция с языковой моделью реализована через тонкий HTTP-клиент без зависимости от специализированных Go-библиотек для работы с LLM — это даёт возможность безболезненно заменить Ollama на любой OpenAI-совместимый API. Во-вторых, rate limit реализован средствами SQL-фильтрации существующих записей в PostgreSQL, что исключает необходимость дополнительного компонента инфраструктуры (Redis, in-memory кэш с истечением). В-третьих, все операции SendMessage выполняются в рамках единого gRPC-запроса с возможностью отмены через context, что обеспечивает корректное освобождение ресурсов при обрыве соединения вышестоящего сервиса.

2.3 Интеграция компонентов системы

Описанный в разделах 2.1–2.2 AI-сервис проектировался для встраивания в произвольную систему управления заявками. В данном разделе формализуется внешний интеграционный контракт сервиса, а затем на примере комплекса Helperbot демонстрируется его использование: сначала со стороны прокси-слоя в helperbot-core (компонент, непосредственно вызывающий AI-сервис по gRPC), затем со стороны Telegram-бота helperbot-tg (компонент, иницирующий цепочку обращений и обрабатывающий ошибки). Реализация сервисов helperbot-core и helperbot-tg выходит за рамки настоящей работы и описывается в объёме, необходимом для верификации корректности интеграционного контракта.

Интеграционный контракт AI-сервиса

Внешний интерфейс `helperbot-ai` определяется gRPC-сервисом `ChatService`, описанным в файле `api/chat/chat_service.proto`. Контракт состоит из двух RPC-методов и четырёх типов сообщений (Листинг 2.9).

Листинг 2.9: Контракт `ChatService` в `api/chat/chat_service.proto`

```
service ChatService {
  rpc NewChat(NewChatRequest) returns (NewChatResponse);
  rpc SendMessage(SendMessageRequest) returns (SendMessageResponse);
}

message NewChatRequest {
  string user_id = 1 [(validate.rules).string.uuid = true];
}

message NewChatResponse {
  string chat_id = 1 [(validate.rules).string.uuid = true];
}

message SendMessageRequest {
  string user_id = 1 [(validate.rules).string.uuid = true];
  string chat_id = 2 [(validate.rules).string.uuid = true];
  string content = 3 [(validate.rules).string = {max_len: 2000}];
}

message SendMessageResponse {
  string chat_id = 1 [(validate.rules).string.uuid = true];
  string content = 2;
}
```

Контракт обладает тремя свойствами, делающими его пригодным для интеграции с произвольной системой-хостом.

Минимальность. Сервис требует от клиента передать только три значения — идентификатор пользователя, идентификатор чата и текст сообщения. Никаких сведений о роли пользователя, его принадлежности к подразделению, формате тикета и других понятиях системы-хоста контракт не использует.

Декларативная валидация. Все ограничения на входные данные (UUID v4 для идентификаторов, максимум 2 000 символов для текста) описаны прямо в `.proto`-файле через правила `protoc-gen-validate` [11]; сгенерированный метод `req.Validate()` вызывается в каждом обработчике до делегирования в сервисный слой (см. раздел 2.1), что исключает попадание некорректных данных в бизнес-логику и проброс их в языковую модель.

Типизированные ошибки. Превышение лимита запросов возвращается через ошибку `model.ErrTooManyRequests`, обёрнутую через директиву `%w` поверх базовой ошибки `ErrPermissionDenied`. Это позволяет вызывающей стороне идентифицировать ситуацию средствами стандартного механизма `errors.Is`, не разбирая текст ошибки, и сформировать понятное пользователю сообщение в UI вместо технической трассы.

Использование контракта на стороне `helperbot-core`

`helperbot-core` реализует собственный `ChatService` с теми же методами `NewChat` и `SendMessage`. Его реализация является тонким прокси: сервисный слой немедленно делегирует вызов репозиторию `chat_backend`, который, в свою очередь, обращается к `helperbot-ai` по gRPC. Установка соединения выполняется при старте сервиса один раз; адрес AI-сервиса задаётся переменной окружения `AI_SERVICE_ADDR` (значение в `Docker Compose` — `helperbot-ai:50052`). Поскольку межсервисный трафик не выходит за пределы изолированной `Docker`-сети, соединение устанавливается без TLS (`insecure.NewCredentials()`).

Ключевую роль играет репозиторий `chat_backend` в `helperbot-core`, конвертирующий доменные типы системы-хоста (`model.UserID`, `model.AiChatID`) в `proto`-сообщения контракта AI-сервиса и обратно. Листинг 2.10 демонстрирует фрагмент реализации, показывающий направление трансляции.

Листинг 2.10: Фрагмент репозитория `chat_backend` в `helperbot-core`

```
func (r *Repository) SendMessage(
    ctx context.Context,
    userId model.UserID,
    chatId model.AiChatID,
    content string,
) (model.AiMessage, error) {
    resp, err := r.chatService.SendMessage(ctx,
        &desc.SendMessageRequest{
            UserId:  userId.String(),
            ChatId:  chatId.String(),
            Content: content,
        })
    if err != nil {
        return model.AiMessage{},
            fmt.Errorf("send message: %w", err)
    }
    chatId, _ = model.NewAiChatID(resp.GetChatId())
    return model.AiMessage{
        ChatID:  chatId,
```

```
        Content: resp.GetContent(),
    }, nil
}
```

Из листинга видно, что helperbot-core полностью инкапсулирует существование отдельного AI-сервиса: для helperbot-tg весь AI-функционал выглядит как часть единого gRPC-сервиса helperbot-core. Если в будущем AI-сервис будет заменён другой реализацией (например, при переходе на облачный LLM-API), потребуется изменить только этот один репозиторий.

Использование контракта на стороне helperbot-tg

Telegram-бот helperbot-tg реализует разветвлённый сценарий взаимодействия с пользователем через конечный автомат (FSM) на основе in-memory кэша. Каждый пользователь идентифицируется по TelegramID, а его текущее состояние хранится в структуре UserState с полем PendingAction. Для AI-чата выделены два состояния (Таблица 2.2).

Таблица 2.2 – Состояния FSM, связанные с AI-чатом

Состояние	Значение	Поведение при получении сообщения
PendingActionGetAiMessage	Ожидание вопроса от пользователя	Сообщение отправляется в AI, состояние → PendingActionWaitAiAnswer
PendingActionWaitAiAnswer	Ожидания ответа от модели	Пользователю сообщается, что запрос обрабатывается

Переход в режим AI-чата инициируется нажатием кнопки «Чат с ИИ»: бот переводит пользователя в состояние PendingActionGetAiMessage и отправляет приветственное сообщение. После этого все последующие текстовые сообщения пользователя маршрутизируются в обработчик processChatMessage, ключевые фрагменты которого с точки зрения интеграционного контракта AI-сервиса представлены в Листинге 2.11.

Листинг 2.11: Фрагмент обработчика AI-сообщений в helperbot-tg, демонстрирующий обработку ошибок интеграционного контракта

```
h.cache.SetPendingAction(telegramID, model.PendingActionWaitAiAnswer)
```

```

ans, err := h.chatService.SendMessage(ctx, telegramID, message)
if err != nil {
    if errors.Is(err, model.ErrTooManyAiRequests) {
        h.bot.Send(c.Chat(), "Слишком много запросов, попробуйте позже")
        h.cache.SetPendingAction(telegramID, model.PendingActionGetAiMessage)
        return nil
    }
    return h.processError(c, newCustomMessageError(
        "Мы не смогли обработать ваш запрос, попробуйте позже", err))
}

h.cache.SetPendingAction(telegramID, model.PendingActionGetAiMessage)
h.bot.Send(c.Chat(), ans.Content)

```

Фрагмент демонстрирует два аспекта корректного использования интеграционного контракта AI-сервиса. Во-первых, ошибка переполнения лимита идентифицируется средствами `errors.Is`, а не сравнением текста сообщения, — это работает благодаря тому, что AI-сервис оборачивает ошибку через директиву `%w` (см. подраздел «Интеграционный контракт»). Во-вторых, при любом исходе вызова состояние FSM возвращается в `PendingActionGetAiMessage`, что позволяет пользователю повторить попытку без перезапуска сценария. Дополнительная проверка состояния перед сменой на `WaitAiAnswer` (опущена в листинге) защищает от гонки: если пока горутина ожидала ответа от языковой модели, пользователь нажал «Отмена», состояние уже сброшено, и ответ молча отбрасывается.

Сервисный слой `helperbot-tg` отвечает за разрешение `TelegramID` в `UserID` и за управление `ChatID` через кэш. Особенностью реализации является ленивое создание чата: при первом обращении пользователя `ChatID` отсутствует в кэше, поэтому вызывается `NewChat` в `helperbot-core` (который проксирует вызов в AI-сервис), а полученный идентификатор сохраняется в кэше для всех последующих сообщений. При перезапуске `helperbot-tg` кэш сбрасывается, и следующий запрос пользователя создаёт новый чат — но история предыдущих диалогов остаётся сохранённой в базе данных `helperbot-ai`. Такой подход избавляет AI-сервис от необходимости хранить пользовательские сессии и допускает горизонтальное масштабирование Telegram-бота.

Меню и доступ к AI-чату по ролям

Кнопка «Чат с ИИ» отображается в меню всех четырёх ролей системы-хоста: корневого администратора, администратора, инженера и библиотекаря. Перед

выполнением действия обработчик вызывает `checkPermission`, которая получает пользователя по TelegramID из `helperbot-core` и сверяет его роль с допустимыми. Листинг 2.12 демонстрирует фрагмент маршрутизации.

Листинг 2.12: Маршрутизация команды AI-чата в `internal/handler/handler.go`

```
case model.TextBtnStartChat:
    if err := h.checkPermission(ctx, c,
        model.UserRoleAdmin,
        model.UserRoleRootAdmin,
        model.UserRoleEngineer,
        model.UserRoleLibrarian,
    ); err != nil {
        return err
    }
    return h.startChatMessage(c)
```

Существенно, что роли пользователей являются понятием системы-хоста и не передаются в AI-сервис: `helperbot-ai` вообще не оперирует ролями. Любая авторизация — обязанность вызывающей стороны. Такое разделение ответственности позволяет интегрировать AI-сервис с системой, использующей произвольную модель прав доступа.

Таким образом, AI-сервис `helperbot-ai` встроен в комплекс `Helperbot` через цепочку из трёх типизированных переходов: `helperbot-tg` → `helperbot-core` (gRPC :50051) → `helperbot-ai` (gRPC :50052) → `Ollama` (HTTP :11434). На стороне `helperbot-core` реализован прокси-репозиторий `chat_backend`, на стороне `helperbot-tg` — конечный автомат с двумя состояниями AI-чата и корректная обработка ошибки превышения лимита через `errors.Is`. Все три аспекта интеграции — клиентский прокси, состояния FSM и обработка типизированных ошибок — определяются исключительно интерфейсом `ChatService` и не требуют от AI-сервиса знаний о внутренней модели системы-хоста.

2.4 Тестирование AI-сервиса

Для верификации корректности ключевых компонентов `helperbot-ai` разработан набор юнит-тестов, охватывающих доменную модель и бизнес-логику сервисного слоя. Тестирование сервисов `helperbot-core` и `helperbot-tg` выходит за рамки настоящей работы. Тесты реализованы на стандартном пакете `testing` языка Go без сторонних зависимостей для мокирования: тестовые заглушки написаны

вручную на основе уже определённых интерфейсов репозитория (см. раздел 2.1), что соответствует идиоматическому подходу к тестированию в Go.

Стратегия тестирования

Юнит-тесты изолируют проверяемый компонент от всех внешних зависимостей: PostgreSQL и Ollama заменяются заглушками (mock), реализующими те же интерфейсы PgRepository и ApiRepository. Заглушки хранят функциональные поля (listMessagesFunc, createMessageFunc, sendMessageFunc), которые переопределяются в каждом тесте для воспроизведения нужного сценария. Такой подход позволяет точно контролировать поведение зависимостей и верифицировать реакцию сервиса на различные ситуации без запуска реальной инфраструктуры. Листинг 2.13 демонстрирует структуру заглушки mockPgRepository.

Листинг 2.13: Заглушка mockPgRepository в internal/service/chat/v1/mock_test.go

```
type mockPgRepository struct {
    listMessagesFunc func(ctx context.Context,
        f model.MessageFilter) ([]model.Message, error)
    createChatFunc func(ctx context.Context,
        userID model.UserID, chatID model.ChatID) error
    createMessageFunc func(ctx context.Context,
        message model.Message) error
}

func (m *mockPgRepository) ListMessages(
    ctx context.Context, f model.MessageFilter,
) ([]model.Message, error) {
    if m.listMessagesFunc != nil {
        return m.listMessagesFunc(ctx, f)
    }
    return nil, nil
}
```

Тесты доменной модели

Пакет internal/model содержит 6 тестов, проверяющих конструкторы типизированных идентификаторов ChatID и UserID. Для каждого типа верифицируются три сценария: успешное создание из корректного UUID, ошибка при пустой строке и ошибка при некорректном формате. Таблица 2.3 содержит перечень тестов и проверяемые условия.

Таблица 2.3 – Тесты доменной модели

Тест	Входные данные	Ожидаемый результат
TestNewChatID_Valid	Корректный UUID	Возврат ChatID, err == nil
TestNewChatID_Empty	Пустая строка	err != nil
TestNewChatID_InvalidUUID	"not-a-uuid"	err != nil
TestNewUserID_Valid	Корректный UUID	Возврат UserID, err == nil
TestNewUserID_Empty	Пустая строка	err != nil
TestNewUserID_InvalidUUID	"invalid"	err != nil

Тесты сервисного слоя

Пакет `internal/service/chat/v1` содержит 4 теста метода `SendMessage`, покрывающих все ветви логики.

`TestSendMessage_RateLimitExceeded` воспроизводит ситуацию, когда пользователь исчерпал лимит запросов. Заглушка `PgRepository` возвращает при фильтрации по `UserID` ровно столько сообщений, сколько задано в конфигурации как максимум. Тест проверяет, что метод возвращает `model.ErrTooManyRequests` через `errors.Is`, то есть ошибка корректно оборачивается по всей цепочке вызовов. Листинг 2.14 демонстрирует его реализацию.

Листинг 2.14: Тест rate limit в `internal/service/chat/v1/send_message_test.go`

```
func TestSendMessage_RateLimitExceeded(t *testing.T) {
    userID := mustUserID(t, "550e8400-e29b-41d4-a716-446655440000")
    chatID := mustChatID(t, "660e8400-e29b-41d4-a716-446655440001")

    pg := &mockPgRepository{
        listMessagesFunc: func(_ context.Context,
            f model.MessageFilter) ([]model.Message, error) {
            if f.UserID != "" {
                return []model.Message{
                    {Role: model.User, Content: "msg1"},
                    {Role: model.User, Content: "msg2"},
                }, nil
            }
            return nil, nil
        },
    },
```

```

    }
    svc := makeService(pg, &mockApiRepository{}, 2)

    _, err := svc.SendMessage(context.Background(),
        userID, chatID, "новый вопрос")
    if !errors.Is(err, model.ErrTooManyRequests) {
        t.Errorf("expected ErrTooManyRequests, got: %v", err)
    }
}

```

TestSendMessage_Success проверяет полный успешный сценарий. Заглушка PgRepository возвращает непустую историю диалога при запросе по ChatID. Заглушка ApiRepository возвращает фиксированный ответ ассистента. Тест верифицирует: возвращённый контент соответствует ответу модели; ChatID в результате равен переданному; в репозиторий было сделано ровно два вызова CreateMessage — для сообщения пользователя и для ответа ассистента; первое сохранённое сообщение имеет роль user, второе — assistant.

TestSendMessage_OllamaError проверяет, что ошибка от ApiRepository оборачивается и возвращается вызывающей стороне без паники и без попытки сохранить сообщения в базу данных.

TestSendMessage_ContextPassedToOllama верифицирует корректную сборку контекста диалога. Заглушка ApiRepository перехватывает переданную структуру model.Chat и сохраняет её для последующей проверки. Тест убеждается, что длина среза сообщений равна длине истории плюс один (новое сообщение пользователя) и что последний элемент среза содержит именно текущий вопрос с ролью user. Данный тест критически важен: неправильная сборка контекста привела бы к тому, что модель отвечала бы без учёта предыдущих вопросов.

Результаты

Все 10 тестов успешно проходят. Сводная таблица тестовых сценариев представлена в таблице 2.4.

Таблица 2.4 – Результаты юнит-тестирования

Тест	Пакет	Что проверяется	Результат
TestNewChatID_Valid	model	Создание ChatID из корректного UUID	PASS
TestNewChatID_Empty	model	Ошибка при пустой строке	PASS
TestNewChatID_InvalidUUID	model	Ошибка при некорректном UUID	PASS
TestNewUserID_Valid	model	Создание UserID из корректного UUID	PASS
TestNewUserID_Empty	model	Ошибка при пустой строке	PASS
TestNewUserID_InvalidUUID	model	Ошибка при некорректном UUID	PASS
TestSendMessage_RateLimitExceeded	service/chat/v1	Возврат ErrTooManyRequests при исчерпании лимита	PASS
TestSendMessage_Success	service/chat/v1	Успешный сценарий: история, ответ, сохранение	PASS
TestSendMessage_OllamaError	service/chat/v1	Ошибка API-репозитория возвращается вызывающей стороне	PASS
TestSendMessage_ContextPassedToOllama	service/chat/v1	Полный контекст диалога передаётся в языковую модель	PASS

2.5 Развёртывание и эксплуатация

Развёртывание всего программного комплекса Helperbot автоматизировано средствами Docker Compose [23] и GNU Make. Единственный файл docker-compose.yml описывает полную инфраструктуру: базы данных, прикладные сервисы и языковую модель. В данном разделе описывается состав инфраструктуры, порядок запуска и процедура первоначальной настройки.

Состав инфраструктуры

Система включает восемь контейнеров, объединённых в изолированную Docker-сеть helperbot-network с драйвером bridge. Полный перечень сервисов и их назначение представлены в таблице 2.5.

Таблица 2.5 – Сервисы Docker Compose

Контейнер	Образ / сборка	Порты	Назначение
postgresql-master	bitnami/postgresql:16	5432	Мастер-узел БД helperbot-core
postgresql-slave	bitnami/postgresql:16	5442	Реплика БД helperbot-core
postgres-ai-master	bitnami/postgresql:16	5433	Мастер-узел БД helperbot-ai
postgres-ai-slave	bitnamilegacy/postgresql:16	5443	Реплика БД helperbot-ai
db-migrations	golang:alpine	—	Применение миграций goose
helperbot-core	Сборка из исходников	8081, 50051	Сервис бизнес-логики
helperbot-tg	Сборка из исходников	—	Telegram-бот
helperbot-ai	Сборка из исходников	8082, 50052	AI-сервис
ollama	ollama/ollama:latest	11434	Среда выполнения LLM

Репликация PostgreSQL

Для каждой из двух баз данных развёртывается пара мастер–реплика на базе образа Bitnami PostgreSQL 16 [20]. Мастер-узел настраивается с параметрами POSTGRESQL_SYNCHRONOUS_COMMIT=on и POSTGRESQL_NUM_SYNCHRONOUS_REPLICAS=1, что обеспечивает синхронную запись: транзакция считается завершённой только после подтверждения от реплики. Листинг 2.15 демонстрирует конфигурацию мастер-узла БД helperbot-ai.

Листинг 2.15: Конфигурация мастер-узла postgres-ai-master в docker-compose.yml

```

postgres-ai-master:
  image: bitnamilegacy/postgresql:16
  container_name: postgres-ai-master
  ports:
    - '5433:5432'
  volumes:
    - postgres_ai_master_data:/bitnami/postgresql
  environment:
    - POSTGRESQL_REPLICATION_MODE=master
    - POSTGRESQL_REPLICATION_USER=repl_user
    - POSTGRESQL_REPLICATION_PASSWORD=repl_password
    - POSTGRESQL_USERNAME=postgres
    - POSTGRESQL_PASSWORD=helperbot
    - POSTGRESQL_DATABASE=postgres
    - POSTGRESQL_SYNCHRONOUS_COMMIT=on
    - POSTGRESQL_NUM_SYNCHRONOUS_REPLICAS=1
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - helperbot-network

```

Реплика-узел подключается к мастеру через переменные `POSTGRESQL_MASTER_HOST` и `POSTGRESQL_MASTER_PORT_NUMBER`. Healthcheck на основе `pg_isready` используется другими сервисами для определения готовности базы данных перед стартом: `helperbot-core` и `helperbot-ai` запускаются только после того, как соответствующие мастер-узлы пройдут проверку работоспособности.

Применение миграций

Миграции схемы базы данных применяются контейнером `db-migrations`, который запускается однократно (`restart: "no"`) после готовности соответствующего мастер-узла. Контейнер устанавливает утилиту `goose` через `go install` и последовательно применяет миграции из директории `/migrations`. В `Makefile` предусмотрены отдельные цели для применения и отката миграций каждой базы данных, что позволяет управлять схемой `helperbot-ai` независимо от `helperbot-core`.

Миграции базы данных `helperbot-ai` состоят из трёх файлов. Первая миграция создаёт таблицу `chats` с первичным ключом `id` `UUID`, полем `user_id` `UUID` и временной меткой `created_at`. Вторая создаёт таблицу `messages` с полями `chat_id`, `role`, `content` и `created_at`; на этапе первоначального проектирования поле `role`

было ошибочно объявлено как UUID. Третья миграция исправляет эту ошибку, изменяя тип столбца `role` с `UUID` на `TEXT`, что обеспечивает прямую сериализацию констант `model.User ("user")` и `model.Assistant ("assistant")` без таблицы-справочника. Откат третьей миграции реализован как `best-effort` — существующие строковые значения не будут валидными `UUID`, что отражено в комментарии секции `goose Down`.

Описание миграций базы данных `helperbot-core` (`departments`, `users`, `requests`, `passwords` и их последующие альтерации) выходит за рамки настоящей работы.

Языковая модель

Создание кастомной модели `support-bot` в `Ollama` выполняется автоматически командой `make compose-up`. Листинг 2.16 демонстрирует соответствующий фрагмент `Makefile`.

Листинг 2.16: Создание модели в `Ollama` в `Makefile`

```
.PHONY: compose-up
compose-up: update-repo
    $(COMPOSE) up -d --build
    docker cp Support.Modelfile ollama:/tmp/Support.Modelfile
    docker exec ollama ollama show support-bot \
        >/dev/null 2>&1 \
        || docker exec ollama ollama create support-bot \
            -f /tmp/Support.Modelfile
```

После старта всех контейнеров `Makefile` копирует `Support.Modelfile` внутрь контейнера `ollama` и проверяет, существует ли модель `support-bot` командой `ollama show`. Если модель отсутствует, она создаётся командой `ollama create`. Условие `||` гарантирует, что повторный запуск `make compose-up` не пересоздаёт модель, если она уже существует, экономя время и вычислительные ресурсы.

`Ollama`-контейнер настраивается с переменной `OLLAMA_KEEP_ALIVE=24h`, удерживающей модель в памяти на 24 часа после последнего обращения, и ограничением памяти в 4 096 МБ, предотвращающим избыточное потребление ресурсов хост-машины.

Конфигурация через `env`-файлы

Каждый прикладной сервис получает конфигурацию через монтируемый `env`-файл. В `Docker Compose` используются `production`-файлы (`.env.helperbot-core`, `.env.helperbot-tg`, `.env.helperbot-ai`), а для локальной разработки

предназначены файлы с суффиксом `.local`, перекрывающие значения через `godotenv.Overload`. Структура адресов в `docker`-сети использует DNS-имена контейнеров: `helperbot-core:50051`, `helperbot-ai:50052`, `ollama:11434`. Листинг 2.17 демонстрирует пример `production`-конфигурации `helperbot-ai`.

Листинг 2.17: Файл `.env.helperbot-ai.example`

```
WRITE_DSN=postgres://postgres:helperbot@postgres-ai-master:5432/postgres?sslmode=disable

GRPC_ADDR=helperbot-ai:50052
HTTP_ADDR=helperbot-ai:8082

REQUESTS_COUNT=10
REQUESTS_REFRESH_INTERVAL=300

AI_ADDR=ollama:11434
```

Порядок запуска и управление

Целевые команды `Makefile` для управления комплексом представлены в таблице 2.6.

Таблица 2.6 – Команды управления инфраструктурой

Команда	Действие
<code>make compose-up</code>	Клонировать/обновить репозитории, собрать образы, запустить все контейнеры, создать модель в Ollama
<code>make compose-start</code>	Запустить остановленные контейнеры без пересборки
<code>make compose-stop</code>	Остановить все контейнеры
<code>make migrate-up</code>	Обновить репозитории и применить все миграции
<code>make migrate-up-ai</code>	Применить миграции только для <code>helperbot-ai</code>
<code>make migrate-down-ai</code>	Откатить последнюю миграцию <code>helperbot-ai</code>
<code>make update-repo</code>	Клонировать или выполнить <code>git pull</code> для всех трёх сервисов

Схема запуска при первом развёртывании

Для первоначального развёртывания системы на сервере достаточно выполнить три шага: создать env-файлы по шаблонам .example, заполнив TG_TOKEN и ROOT_ADMIN_ID; запустить `make compose-up`; дождаться завершения `healthcheck` всех узлов PostgreSQL. Docker Compose самостоятельно соблюдает порядок зависимостей: сначала стартуют мастер-узлы БД, затем применяются миграции, затем запускаются прикладные сервисы.

Таким образом, развёртывание AI-сервиса `helperbot-ai` вместе со всеми его зависимостями автоматизировано и выполняется единственной командой `make compose-up`. Развёртывание сопутствующих сервисов `helperbot-core` и `helperbot-tg`, обеспечивающих интеграционный контекст AI-сервиса, осуществляется тем же `docker-compose.yml` без дополнительных действий со стороны оператора.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы разработан микросервис `helperbot-ai` — AI-сервис автоматической обработки пользовательских обращений на основе локальной языковой модели Qwen2.5 под управлением Ollama. Сервис встроен в программный комплекс `Helperbot` и апробирован в применении к Централизованной библиотечной системе Петроградского района Санкт-Петербурга.

Основные результаты работы по поставленным задачам:

1. Анализ предметной области — сравнение пяти распространённых решений (OpenAI Assistants API, Yandex GPT API, Rasa, Botpress, Google Dialogflow CX, таблица 1.1) обосновало разработку собственного сервиса на базе локальной языковой модели под управлением Ollama.
2. Требования — сформулированы 6 функциональных и 8 нефункциональных требований к AI-сервису с количественными ограничениями (10 запросов за 300 с, 2 000 символов на сообщение, таймаут LLM 60 с), закреплёнными в контракте `proto-gen-validate` и в конфигурации сервиса.
3. Архитектура — определён минимальный интеграционный контракт AI-сервиса из двух gRPC-методов (`NewChat`, `SendMessage`) и схема собственной базы данных из двух таблиц (`chats`, `messages`), сознательно не содержащая сущностей системы-хоста, что обеспечивает встраиваемость сервиса в произвольную систему управления заявками.
4. Реализация — микросервис реализован на Go по многослойной архитектуре с инверсией зависимостей через `Uber FX`; ограничение частоты запросов реализовано SQL-фильтрацией без внешних компонентов; кастомная модель `support-bot` на базе `qwen2.5:1.5b` настроена через `Modelfile` с русскоязычным системным промптом.
5. Интеграция — на стороне `helperbot-core` реализован прокси-репозиторий `chat_backend`, на стороне `helperbot-tg` — конечный автомат с двумя состояниями AI-

чата; типизированная ошибка `ErrTooManyRequests` распознаётся клиентом средствами `errors.Is` без разбора текста сообщения.

6. Тестирование и развёртывание — разработано 10 юнит-тестов, охватывающих все ветви бизнес-логики `SendMessage` (все `PASS`); развёртывание комплекса автоматизировано средствами `Docker Compose` и `Makefile` и выполняется единственной командой `make compose-up`.

Сформулированная во введении цель достигнута, все поставленные задачи решены.

Практическая ценность разработанного AI-сервиса состоит в готовности к встраиванию в произвольную систему управления заявками через минимальный gRPC-контракт без зависимости от внешних AI-провайдеров и без передачи пользовательских данных за пределы инфраструктуры организации. Эти свойства делают сервис применимым в средах с жёсткими требованиями к информационной безопасности — государственных учреждениях, образовательных организациях, корпоративных службах поддержки. Апробация в составе комплекса `Helperbot` подтвердила корректность интеграционного контракта на полной цепочке вызовов Telegram-бот → бизнес-логика → AI-сервис → языковая модель.

Направлениями дальнейшего развития AI-сервиса являются: реализация векторного поиска по базе знаний организации (RAG) для повышения точности ответов на специфические вопросы; расширение системного промпта примерами типовых обращений (few-shot prompting); включение чтения исторических сообщений с реплики PostgreSQL для разгрузки мастер-узла; переход на более крупную базовую модель (qwen2.5:7b или сопоставимую) при увеличении доступных вычислительных ресурсов; добавление REST-шлюза через gRPC-gateway для интеграции с системами, не поддерживающими gRPC.

СПИСОК ЛИТЕРАТУРЫ

1. OpenAI Assistants API. — 2026. —
URL: <https://platform.openai.com/docs/assistants/overview> (дата обращения 28.04.2026).
2. Yandex GPT API — документация. — 2026. —
URL: <https://yandex.cloud/ru/docs/foundation-models/concepts/yandexgpt> (дата обращения 28.04.2026).
3. Rasa Open Source — платформа для разработки диалоговых систем. — 2026. —
URL: <https://rasa.com/docs/rasa> (дата обращения 28.04.2026).
4. Botpress — платформа для создания чат-ботов. — 2026. —
URL: <https://botpress.com/docs> (дата обращения 28.04.2026).
5. Google Dialogflow CX — документация. — 2026. —
URL: <https://cloud.google.com/dialogflow/cx/docs> (дата обращения 29.04.2026).
6. Ollama — локальный запуск языковых моделей. — 2026. —
URL: <https://github.com/ollama/ollama> (дата обращения 29.04.2026).
7. The Go Programming Language Specification. — 2026. —
URL: <https://go.dev/ref/spec> (дата обращения 01.05.2026).
8. gRPC — A high performance, open source universal RPC framework. — 2026. —
URL: <https://grpc.io/docs> (дата обращения 01.05.2026).
9. Protocol Buffers Documentation. — 2026. —
URL: <https://protobuf.dev/overview> (дата обращения 01.05.2026).
10. grpc-gateway — gRPC to JSON proxy generator. — 2026. —
URL: <https://github.com/grpc-ecosystem/grpc-gateway> (дата обращения 02.05.2026).
11. protoc-gen-validate — Protobuf validation library. — 2026. —
URL: <https://github.com/bufbuild/protoc-gen-validate> (дата обращения 02.05.2026).
12. PostgreSQL 16 — документация. — 2026. —
URL: <https://www.postgresql.org/docs/16> (дата обращения 03.05.2026).
13. pgx — PostgreSQL driver and toolkit for Go. — 2026. —
URL: <https://github.com/jackc/pgx> (дата обращения 03.05.2026).

14. Uber FX — Dependency injection framework for Go. — 2026. — URL: <https://github.com/uber-go/fx> (дата обращения 05.05.2026).
15. go-grpc-middleware — gRPC interceptors for Go. — 2026. — URL: <https://github.com/grpc-ecosystem/go-grpc-middleware> (дата обращения 05.05.2026).
16. Qwen2.5: A Party of Foundation Models. — 2026. — URL: <https://qwenlm.github.io/blog/qwen2.5> (дата обращения 06.05.2026).
17. telebot.v4 — Telegram Bot framework in Go. — 2026. — URL: <https://github.com/tucnak/telebot> (дата обращения 07.05.2026).
18. Goose — database migration tool for Go. — 2026. — URL: <https://github.com/pressly/goose> (дата обращения 07.05.2026).
19. Docker Compose — документация. — 2026. — URL: <https://docs.docker.com/compose> (дата обращения 08.05.2026).
20. Bitnami PostgreSQL Docker Image. — 2026. — URL: <https://github.com/bitnami/containers/tree/main/bitnami/postgresql> (дата обращения 08.05.2026).
21. Стоянов Ж., Христоский И., Стоянова Е., Стойкова А. Направления будущих исследований и рекомендации по развитию микросервисной архитектуры // Труды Института системного программирования РАН. — 2024. — Т. 36, № 1. — С. 105–130. — DOI: 10.15514/ISPRAS-2024-36(1)-7. — URL: <https://ispranproceedings.elpub.ru/jour/article/view/1760> (дата обращения 10.05.2026).
22. Андрианов А. В., Калинин А. С., Лаптева М. Г. Использование gRPC в современных микросервисных архитектурах: плюсы и минусы по сравнению с REST // КиберЛенинка. — 2025. — URL: <https://cyberleninka.ru/article/n/ispolzovanie-grpc-v-sovremennyh-mikroservisnyh-arhitekturah-plyusy-i-minusy-po-sravneniyu-s-rest> (дата обращения 10.05.2026).
23. Чиганов Д. Р. Docker: ключ к контейнеризации и масштабируемости // КиберЛенинка. — 2023. — URL: <https://cyberleninka.ru/article/n/docker-klyuch-k-konteynerizatsii-i-masshtabiruemosti> (дата обращения 11.05.2026).
24. Запатоцкий Ю. М. Исследование применения больших языковых моделей для повышения производительности работы второй линии техподдержки : магистерская диссертация / Уральский федеральный университет. —

Екатеринбург, 2025. — 86 с. — URL: <https://elar.urfu.ru/handle/10995/147361> (дата обращения 12.05.2026).